

OpenMath Agent

실행 기반 감사 리포트 v2 (보강판)

기존 코드 수정 없이 · 에이전트 각각 직접 실행 · 부족한 부분 정밀 파악

감사 일자: 2026-05-28 · 감사 범위: agents/ steps/ tools/ policies/ workflows/ · 버전: v2 (보강 5 항목)

한 줄 결론: agent 21개 진입점 중 17개 stub (throw). 동작하는 건 RAG client + Zod 스키마 + 불변식 5개뿐. 외부 서비스(cliproxy, math-engine)는 둘 다 살아있음 — wrapper(연결 다리)만 빠진 상태. **추가로 이미 구현된 코드에서도 약점 4건 발견.**

0 먼저 이해하기 — stub이 뭔가?

코드가 있는데 왜 안 돌아가는지 이해하는 게 핵심

stub(스텝)이란?

코드 파일은 있고 함수 이름도 있는데, 안에 실제 내용이 없고 '나중에 만들어줘' 라고 적어둔 상태야. 마치 건물 외벽과 뼈대는 다 있는데 내부 공사가 하나도 안 된 것처럼.

stub 코드 예시

함수 이름: `createGeneratorAgent()`

내부 내용: `throw new Error('not implemented yet')`

→ 호출하면 즉시 오류 발생. 아무것도 안 함.

OpenMath agent의 현재 상태

설계도(인터페이스, 타입, 스키마)는 완성되어 있어. 하지만 실제로 일하는 코드 본문이 아직 채워지지 않았어. 뼈대 100%, 내용물 0% 인 상태야.

영역	상태	쉽게 말하면
설계도 (타입·스키마)	✓ 완성	어떻게 만들지 설계는 다 됨
프롬프트 파일 6개	✓ 완성	AI한테 줄 지시문은 다 있음
외부 서비스 연결	✓ 살아있음	cliproxy, math-engine 둘 다 켜져있음
RAG 검색 엔진	✓ 동작	기출문제 검색 기능은 구현됨
agent 4개 본문	✗ 0/4 stub	AI 호출하는 핵심 코드 없음
step 6개 본문	✗ 0/6 stub	6단계 처리 코드 없음
tool 3개 본문	✗ 0/3 stub	LLM·math·프롬프트 연결 코드 없음
policy 3개 본문	✗ 0/3 stub	재시도·타임아웃·판정 코드 없음

1 사전 체크 — 실행 환경 확인

코드 돌리기 전에 필요한 서비스들이 켜져있는지 확인

확인 항목	결과	의미
cliproxy (127.0.0.1:8317)	✓ 응답	Claude/GPT 연결 서버 켜져있음
math-engine (localhost:8000)	✓ 응답	수학 계산기 서버 켜져있음
packages/agent/.env 파일	✗ 없음	.env.example만 있음 → 직접 만들어야 함
pnpm install (패키지)	✓ 완료	필요한 도구 다 설치됨
기존 코드 수정	✓ 0줄	scripts/ 에만 테스트 파일 추가

주의: .env 파일 없음 → scripts/ 안에서 환경변수를 직접 주입하는 방식으로 우회해서 실행함

v2 추가 감사 작업이 코드를 깨뜨렸는지 검증 — baseline 3/3 통과

scripts/ 폴더에 진단 파일 8개를 추가했는데, 그게 기존 빌드를 깨뜨리지 않았는지 확인.

검사	결과	상세
pnpm typecheck	✓ exit 0	TypeScript 타입 에러 0건
pnpm test	✓ 7/7 통과	2 files (rag-client 2건 + placeholder 5건)
pnpm build	✓ 성공	dist/index.js 12.64KB + .d.ts 85.46KB (72ms ESM + 4.3s DTS)

핵심: 감사용 스크립트 8개 추가가 기존 코드를 1줄도 깨뜨리지 않음. **destructive 영향 0건 확정.**

2 발견된 문제 — 환경변수 이름 불일치

.env.example 에 적힌 이름과 실제 코드가 인식하는 이름이 달라서 조용히 무시됨

.env.example 파일을 보고 그대로 복사해서 쓰면 코드가 인식을 못 해. 파일에 적힌 이름과 코드가 실제로 읽는 이름이 달라. 오류도 안 나고 그냥 조용히 무시되니까 더 위험해.

.env.example 에 적힌 이름	코드가 실제로 읽는 이름	문제
OPENAI_API_KEY	LLM_API_KEY	이름이 달라 → 조용히 무시됨
CLIPROXY_BASE_URL	LLM_BASE_URL	이름이 달라 → 조용히 무시됨
CLIPROXY_API_KEY	LLM_API_KEY	이름이 달라 → 조용히 무시됨
LLM_PROVIDER=clipproxy	openai-compatible 써야 함	'clipproxy' 값 자체가 거부됨
CLIPROXY_MODEL	코드에 정의 없음	이 변수 자체가 코드에 없음

올바른 .env 설정

```
LLM_PROVIDER=openai-compatible
LLM_BASE_URL=http://127.0.0.1:8317/v1
LLM_API_KEY=my-secret-key
MATH_ENGINE_URL=http://localhost:8000
```

3 발견된 문제 — 프롬프트가 없는 스키마 참조

AI 지시문 파일이 존재하지 않는 데이터 형식을 참조하고 있음

AI한테 보내는 지시문(프롬프트) 파일 6개 중 3개가 아직 만들어지지 않은 데이터 형식을 참조하고 있어. prompt-loader 가 구현되면 이 3개에서 오류가 날 거야.

프롬프트 파일	참조하는 데이터 형식	실제로 있나?
problem-generator.md	GeneratedProblemSchema	✓ 있음
refiner.md	GeneratedProblemSchema	✓ 있음
intent-extraction.md	IntentSchema	✓ 있음
constraint-critic.md	CritiqueSchema	X 없음
independent-solver.md	SolveAttemptSchema	X 없음
objective-mapper.md	ObjectiveMappingNuanceSchema	X 없음 (TODO 명시)

4 P0 도구 3개 실행 결과 (가장 먼저 만들어야 함)

이게 없으면 agent도, step도 아무것도 못 돌아감

P0는 '최우선 순위'라는 뜻이야. 이 3개가 완성되어야 나머지 모든 것이 돌아갈 수 있어. 지금은 3개 모두 stub(껍데기)만 있는 상태야.

[P0-A] LLM 연결 도구 (tools/llm-provider.ts)

이게 하는 일: Claude나 GPT 같은 AI 모델을 실제로 연결하는 도구야. 이게 없으면 어떤 AI도 호출할 수 없어.

실행 결과: **✗ 실패**

오류 발생 위치: tools/llm-provider.ts 24번 줄

오류 메시지: resolveLanguageModel: not implemented yet

cliproxy 연결: **✗ 불가** (연결 도구 자체가 없어서)

[P0-B] 프롬프트 파일 읽기 도구 (tools/prompt-loader.ts)

이게 하는 일: AI한테 줄 지시문(.md 파일)을 읽어서 불러오는 도구야. 이게 없으면 AI가 뭘 해야 할지 지시를 받지 못해.

실행 결과: **✗ 실패**

오류 발생 위치: tools/prompt-loader.ts 42번 줄

오류 메시지: createFsPromptLoader: not implemented yet

로드 성공한 프롬프트: 0개 / 6개

[P0-C] 수학 계산기 연결 도구 (tools/math-engine-client.ts)

이게 하는 일: Python SymPy 수학 계산기 서버에 HTTP로 요청을 보내는 도구야. 수학 서버 자체는 켜져 있고 잘 돌아가는데, Node 쪽에서 연결하는 다리만 없는 상태야.

실행 결과: **⚠ 절반 성공**

오류 발생 위치: tools/math-engine-client.ts 73번 줄

오류 메시지: createMathEngineClient: not implemented yet

math-engine 서버 직접 호출: **✓ 5/5 정상**

- /solve: $x^2-5x+6=0 \rightarrow \{\text{solutions: ['2', '3']}\}$ ✓
- /verify: $(x+1)(x-1) \equiv x^2-1 \rightarrow \{\text{equivalent: true}\}$ ✓
- /simplify, /differentiate, /limit $\rightarrow$ 전부 정상 ✓

핵심 발견: 수학 계산기 서버는 완벽히 작동함. Node 쪽 연결 다리(wrapper)만 만들면 바로 연결 가능!

5 P1 에이전트 4개 실행 결과

실제로 AI를 호출해서 일하는 부품들 — 전부 stub 상태

에이전트는 실제로 AI(Claude/GPT)를 불러서 수학 문제를 만들거나 검토하는 부품이야. 4개 모두 껍데기만 있고 내부 코드가 없어서 호출하면 즉시 오류가 나.

에이전트	하는 일	오류 위치	오류 메시지
Generator Agent	수학 문제 초안 생성	generator-agent.ts:32	not implemented yet
Constraint Critic Agent	조건 위반 검사	constraint-critic-agent.ts:31	not implemented yet
Refiner Agent	위반 부분 수정	refiner-agent.ts:23	not implemented yet
Solver Agent	독립적으로 문제 다시 풀기	solver-agent.ts:24	not implemented yet

v2 추가 추가 발견 — 입력 데이터 형식 자체에도 불일치 5건

사용자가 spec 에 적은 dummy 입력 형식(예: intent.achievementCode) 이 실제 코드 schema(intent.objective_code) 와 다름. factory 가 먼저 throw 라 silent. stub 구현 후 재검증 필요.

사용자 dummy 필드	실제 코드 schema 필드	mismatch 종류
intent.achievementCode	intent.objective_code	이름 다름
intent.evaluationDimensions: string[]	evaluation_dimensions: {id, description, must_preserve}[]	형태 다름 (배열 요소 객체)
intent.requiredTechniques	required_techniques	snake_case 차이
ragResults[].sourceProblem	refs[].problem	키 이름 + 컨테이너
constraints.integerAnswerOnly	(schema 에 없음)	필드 자체 없음

6 P1 단계(Step) 6개 실행 결과

6단계 파이프라인의 각 단계 — 전부 stub 상태

step은 6단계 검증 파이프라인의 각 단계야. 에이전트를 감싸는 얇은 포장지 역할인데, 포장지도 없는 상태야.

단계	하는 일	오류 위치	상태
ragSearch	기출문제 검색	rag-search.ts:22	X stub
extractIntent	의도 분석	intent-extraction.ts:32	X stub
generateProblem	문제 생성	problem-generation.ts:41	X stub
verifyWithSympy	수학 계산기 검증	sympy-verification.ts:22	X stub
independentResolve	독립 재풀이	independent-resolve.ts:26	X stub
mapObjective	학습목표 매핑	objective-mapping.ts:31	X stub

7 v2 추가 P2 정책(Policy) 3개 실행 결과

재시도·타임아웃·최종 판정 정책 — 전부 stub 상태

정책(policy)은 "재시도 몇 번까지 할까", "한 단계 몇 초 안에 끝내야 할까", "6단계 결과를 어떻게 종합해서 verified/rejected/warning 으로 판정할까" 같은 룰을 담당하는 부품이야. 3개 모두 stub.

정책	하는 일	오류 위치	상태
createAcceptancePolicy	6단계 결과 종합 → 최종 판정 (verified/rejected/warning)	acceptance-policy.ts:10	X stub
createBoundedRetryPolicy	실패 시 몇 번까지 재시도할지 결정 (최대 3회, I-V5)	retry-policy.ts:22	X stub
withTimeout	한 단계가 너무 오래 걸리면 끊기 (기본 30초)	timeout-policy.ts:12	X stub

참고: 이 3개는 정책 로직이 단순해서 stub 구현 난이도가 가장 낮음. 그러나 orchestrator (runVerificationWorkflow) 가 이 3개에 의존하므로 우선순위는 후순위.

8 정상 동작 확인 — 불변식 검증

Zod 스키마 + 5개 불변식은 완벽하게 구현되어 있음

불변식이란 '절대 어겨서는 안 되는 규칙'이야. 예를 들어 '수학 계산기가 실패했으면 최종 결과도 실패여야 한다' 같은 규칙. 이 규칙들을 검사하는 코드는 이미 완벽하게 구현되어 있어 — 7가지 케이스 전부 통과.

불변식	검사 내용	결과
I-V1	검증 단계가 정확히 6개인지 확인	✓ 통과
I-V2/V3	수학 계산기 실패 → 최종 결과도 반드시 실패	✓ 통과
I-V4-A	경고 조건이 맞는지 확인 (위반 케이스)	✓ 통과
I-V4-B	경고 조건이 맞는지 확인 (정상 케이스)	✓ 통과
I-V5-A	재시도 4회 이상이면 반드시 실패 처리	✓ 통과
I-V5-B	재시도 4회 + 실패 처리 (정상 케이스)	✓ 통과
정상 케이스	모든 단계 통과 + 최종 검증됨	✓ 통과

핵심 발견: 규칙 검사 코드는 완벽히 동작함. stub 본문이 채워지면 자동으로 규칙이 보장됨!

9

v2 추가

추가 발견 — 이미 구현된 코드의 약점 4건

stub 외에도 "이미 동작하는 코드" 에서 버그/약점 발견

감사 메시지가 "stub만 채우면 끝" 으로 단순화되면 위험해. 실제로 이미 구현된 코드 4건에서도 약점 이 발견 됨. stub 구현 이전/이후 따로 fix 필요.

● 약점 1 — RAG 검색 엔진의 Zod 검증 누락 (D-7 부분 위반)

위치: rag-client.ts:148-182 (toIndexedProblem 함수)

현상: JSONL 기출문제를 메모리에 로드할 때 TypeScript 타입 강제만 있고, **Zod 런타임 검증 (SourceProblemSchema.parse) 호출 없음.**

결과: 깨진 데이터가 들어와도 통과. 예: grade: "abc" (숫자여야 함) 또는 grade: null 도 그대로 통과 → I-S1~I-S5 (SourceProblem 불변식 5건) 강제 안 됨.

권장: SourceProblemSchema.parse(problem) 한 줄 추가.

● 약점 2 — health endpoint 가 가짜 OK 반환 (D-3 위반)

위치: server/routes/health.ts:10

현상: health 핸들러가 _mathEngine 파라미터를 받아놓고 **한 번도 호출하지 않음.** 응답은 항상 {status:"ok"}.

결과: math-engine 서버가 죽어도 agent /health 는 200 OK 반환 → 모니터링이 silent failure 를 못 잡음.

권장: await mathEngine.health() 호출 + 실패 시 503 반환.

● 약점 3 — Windows 에서 main() 호출 안 됨

위치: src/index.ts:14

현상: 진입점 가드 if (import.meta.url === `file://\${process.argv[1]}`) 가 Windows 에서 백슬래시/슬래시 차이로 매번 false. **main() 자체가 호출 안 됨.**

결과: pnpm dev 가 침묵 (에러도 출력 안 됨). 개발자가 "내가 뭘 잘못했나" 헛갈림.

권장: pathToFileURL(process.argv[1]).href 변환 후 비교.

● 약점 4 — 모델 ID 하드코딩 (cliproxy 즉시 fail)

위치: config/models.ts (DEFAULT_MODELS)

현상: 6개 agent 역할의 모델이 전부 gpt-4o / gpt-4o-mini 하드코딩. env 변수로 override 불가능.

결과: cliproxy 모델 목록에 gpt-4o 없음 (cliproxy 는 claude-haiku-4-5, gemini-2.5-flash 등 제공) → 즉시 unknown provider for model gpt-4o 에러.

권장: env 변수 LLM_MODEL_* 로 override 가능하게 변경.

요약: stub 17건 채우는 작업과 별개로, 이미 동작하는 코드 4건에도 fix 필요. 약점 1, 2 는 D-7/D-3 도메인 원칙 위반이라 우선순위 높음.

10

v2 추가

작은 발견 — AGENTS.md 문서 자체의 숫자 오류

감사가 spec 문서의 잘못된 표기까지 잡음

AGENTS.md §9 의 표기: "grep -rn 'not implemented yet' packages/agent/src 로 14곳 확인 가능"

본 감사 실측: grep 결과 정확히 21곳 (P0:3 + P1:11 + P2:4 + P3:3)

차이 7곳의 원인: §9 가 일부 항목을 "agents/*-agent.ts (4개)" 같이 묶음으로 카운트해서 발생. 정수는 21.

권장: AGENTS.md §9 의 "14곳" 표기를 "21곳 (묶음 9 행)" 으로 갱신.